Primer entregable: Plaza U con JavaScript y React

Construcción de Software III · Docente: Argenis Gregorio Gamboa Santos
Integrantes: Miguel Angel Restrepo Pineda y Lucas Lobo Lopez
Tecnológico de Antioquia · Noviembre de 2026
Repositorio: [pegar aquí el enlace del repositorio de GitHub]

1. Descripción del problema

En la universidad hay estudiantes que hacen productos (comida, velas, artesanías, ropa) y otros que saben enseñar algo (programación, música, idiomas). Hoy esas dos actividades se ofrecen por separado, en chats y redes sociales, y no existe un lugar propio donde reunirlas con reglas claras, como entregar solo dentro del campus.

Esto les genera una dificultad a los dos lados. El que vende o enseña tiene que promocionarse por su cuenta, y el que quiere comprar o inscribirse no sabe dónde buscar ni qué ofrece cada uno. Hace falta una plataforma web que reúna ambas actividades en un solo sitio.

Plaza U es esa plataforma: comercializa productos hechos por estudiantes y ofrece cursos presenciales dictados por estudiantes, con entregas y clases dentro de la universidad.

2. Objetivo de este entregable

Iniciar la implementación del proyecto con JavaScript y React. En este primer entregable construimos el **frontend** con datos simulados en arreglos, sin base de datos ni servidor, para validar la arquitectura y el manejo del estado antes de conectar el resto del sistema.

3. Requisitos funcionales

ID	Requisito	Prioridad	Descripción
RF-01	Ver el catálogo	Alta	La persona ve los productos del mercadito en tarjetas, con foto, categoría, vendedor y precio.
RF-02	Buscar, filtrar y ordenar	Alta	Puede buscar por texto, filtrar por categoría y ordenar por precio.
RF-03	Carrito de compras	Alta	Puede agregar productos, cambiar la cantidad, quitarlos, ver el total y finalizar el pedido.
RF-04	Ver y filtrar cursos	Media	Ve los cursos presenciales y los filtra por texto y por nivel.
RF-05	Inscribirse a un curso	Alta	Se inscribe a un curso con cupos disponibles y lo ve en «Mis inscripciones».
RF-06	Publicar un producto	Media	Un estudiante publica un producto con un formulario validado y puede eliminarlo.
RF-07	Navegar entre secciones	Alta	Se mueve entre Inicio, Mercadito, Cursos y Vender desde el menú, sin recargar la página.

4. Especificación de los requisitos principales

RF-02 · Buscar, filtrar y ordenar productos

Elemento	Detalle
Entradas	Texto del buscador, categoría elegida y forma de ordenar.
Proceso	Se filtran los productos que cumplen la categoría y contienen el texto en el nombre, la descripción o la categoría. La búsqueda ignora tildes y mayúsculas. Luego se ordena una copia del resultado.
Salida	Lista de tarjetas actualizada y un contador («3 productos»). Si no hay resultados se muestra un mensaje con un botón para limpiar los filtros.
Criterios de aceptación	1. Escribir «cafe» sin tilde encuentra el café de origen Antioquia. 2. La categoría «Comida» muestra 3 productos. 3. «Precio: menor a mayor» deja los precios en orden ascendente. 4. Una búsqueda sin coincidencias muestra el mensaje vacío.

RF-03 · Carrito de compras

Elemento	Detalle
Entradas	Clic en «Agregar», en los botones de más y menos, en «Quitar» y en «Finalizar pedido».
Proceso	Al agregar se revisa que no se supere el stock. La cantidad se puede subir o bajar, y una línea que llega a cero se elimina. Al finalizar se descuenta el stock de cada producto y se vacía el carrito.
Salida	Contador de unidades en el encabezado, lista de líneas con su subtotal, total del pedido y un aviso de confirmación.
Criterios de aceptación	1. No deja agregar más unidades de las que hay (del oso tejido solo se pueden 6). 2. El total se recalcula con cada cambio (6 osos y 3 cafés suman $ 354.000). 3. «Quitar» elimina la línea. 4. Al finalizar, el carrito queda en cero y el stock baja: el oso pasa a «Agotado». 5. La tecla Escape cierra el carrito.

RF-05 · Inscribirse a un curso

Elemento	Detalle
Entradas	Clic en «Inscribirme» de un curso.
Proceso	Se revisa que la persona no esté inscrita y que queden cupos. Si todo está bien, se descuenta un cupo y se guarda la inscripción.
Salida	El botón cambia a «Inscrito ✓» y se deshabilita, el cupo baja en uno, aparece un aviso y el curso se agrega a «Mis inscripciones».
Criterios de aceptación	1. Al inscribirse, los cupos del curso bajan en 1. 2. No se puede inscribir dos veces al mismo curso. 3. Un curso sin cupos muestra «Sin cupos» y su botón está deshabilitado.

RF-06 · Publicar un producto

Elemento	Detalle
Entradas	Nombre, categoría, precio, unidades disponibles y descripción.
Proceso	Se valida: nombre de al menos 3 letras, precio mínimo de $500, unidades de 0 o más y descripción de al menos 10 letras. Si hay errores, se muestran junto a cada campo y no se publica.
Salida	El producto aparece de primero en el mercadito y en «Mis productos», con un botón para eliminarlo. El formulario se limpia.
Criterios de aceptación	1. El formulario vacío muestra 4 mensajes de error. 2. Con datos válidos, el producto se publica y aparece primero en el mercadito. 3. «Eliminar» lo quita de la lista y del mercadito.

Requisitos no funcionales

- **Usabilidad:** la interfaz se adapta al celular (probada a 390 px de ancho, sin barra de desplazamiento horizontal) y se puede usar con teclado.
- **Mantenibilidad:** el código está dividido en páginas, componentes, datos y utilidades, y se revisa con ESLint y Prettier.
- **Compatibilidad:** funciona en navegadores modernos (Chrome, Edge, Firefox).
- **Alcance de esta versión:** los datos son de ejemplo y se reinician al recargar la página.

5. Arquitectura y estructura del proyecto

Hicimos el proyecto con **React 18** y **Vite**. Toda la aplicación vive en un componente raíz, `App`, que guarda el estado y lo reparte a las páginas y a los componentes por medio de **props**. Para cambiar de sección no usamos un router: guardamos en un estado qué página se ve y mostramos la que corresponde.

```
plaza-u-react/
  index.html          página base
  public/             logo y fotos de ejemplo
  src/
    main.jsx          arranca React
    App.jsx           componente raíz: estado y funciones
    data/             productos, cursos, vendedores y categorías (arreglos)
    pages/            Inicio, Mercadito, Cursos, Vender
    components/       piezas reutilizables
    utils/            formato de pesos, búsqueda por id
    styles/           estilos.css con la paleta
  docs/capturas/      capturas del sistema
```

Los **datos simulados** están en `src/data` como arreglos de objetos. Un producto, por ejemplo, tiene `id`, `nombre`, `categoria`, `precio`, `stock`, `descripcion`, `foto` y `vendedorId`. El vendedor se busca por ese id en el arreglo de vendedores.

6. Componentes implementados

Componente	Qué hace	Props principales
`App`	Guarda el estado (página, productos, cursos, carrito, inscripciones, aviso) y define las funciones que lo cambian.	No recibe props.
`Header`	Logo, menú de navegación (con `map`) y botón del carrito con su contador.	`pagina`, `onNavegar`, `cantidadCarrito`, `onAbrirCarrito`
`Footer`	Pie de página con el eslogan.	Ninguna.
`TarjetaProducto`	Muestra un producto y su botón «Agregar». Cambia a «Agotado» si no hay stock y avisa cuando quedan pocas unidades.	`producto`, `vendedor`, `onAgregar`
`TarjetaCurso`	Muestra un curso. El botón cambia a «Inscrito ✓» o «Sin cupos» según el caso.	`curso`, `instructor`, `inscrito`, `onInscribirse`
`Chips`	Fila de botones para filtrar. Se usa para las categorías y para los niveles.	`opciones`, `seleccionada`, `onSeleccionar`, `textoTodas`
`Buscador`	Campo de búsqueda controlado.	`valor`, `onCambio`, `etiqueta`, `placeholder`
`Carrito`	Panel lateral con las líneas del pedido, las cantidades, el total y el botón para finalizar.	`abierto`, `items`, `onCerrar`, `onCambiarCantidad`, `onQuitar`, `onFinalizar`
`FormularioProducto`	Formulario para publicar un producto, con validación. Guarda sus campos y sus errores en estados propios.	`onPublicar`
`EstadoVacio`	Mensaje cuando una búsqueda no encuentra nada, con botón para limpiar.	`mensaje`, `textoBoton`, `onLimpiar`
`Aviso`	Mensaje breve de confirmación que desaparece solo.	`texto`
`Inicio`	Portada, productos y cursos destacados y los pasos de «Así funciona».	`productos`, `cursos`, `vendedores`, `inscripciones`, `onNavegar`, `onAgregar`, `onInscribirse`
`Mercadito`	Catálogo con buscador, categorías y orden. Tiene estados propios para los filtros.	`productos`, `vendedores`, `onAgregar`

Componente	Qué hace	Props principales
`Cursos`	Lista de cursos con filtros e inscripciones.	`cursos`, `instructores`, `inscripciones`, `onInscribirse`
`Vender`	Formulario para publicar y lista de «Mis productos».	`misProductos`, `onPublicar`, `onEliminar`

7. Cómo se aplican los conceptos de React y JavaScript

Estado con useState

El estado del carrito está en `App`. Cada vez que cambia, React vuelve a dibujar lo que depende de él, como el contador del encabezado y el panel del carrito.

```
const [carrito, setCarrito] = useState([]); // líneas: { id, cantidad }
const [inscripciones, setInscripciones] = useState([]); // ids de los cursos
tomados
```

Listas con map()

Cada lista se dibuja con `map`, una tarjeta por elemento, y cada una lleva una `key` única para que React pueda identificarla.

```
{ordenados.map(producto => (
  <TarjetaProducto key={producto.id} producto={producto}
    vendedor={buscarPorId(vendedores, producto.vendedorId)}
onAgregar={onAgregar} />
))}
```

Eventos

Usamos `onClick` (agregar, quitar, inscribirse, navegar), `onChange` (buscador, orden y campos del formulario), `onSubmit` (publicar un producto) y un evento de teclado, Escape, para cerrar el carrito.

```
<button className="boton boton-verde" disabled={agotado}
  onClick={() => onAgregar(producto.id)}>Agregar</button>
```

Arreglos y objetos

Para no modificar el estado directamente, siempre creamos una copia nueva con `map`, `filter` y el operador de propagación (`...`). Por ejemplo, para subir la cantidad de una línea del carrito: `carrito.map(l => (l.id === id ? { ...l, cantidad: l.cantidad + 1 } : l))`. El total del carrito se calcula con `reduce`.

8. Capturas del sistema funcionando

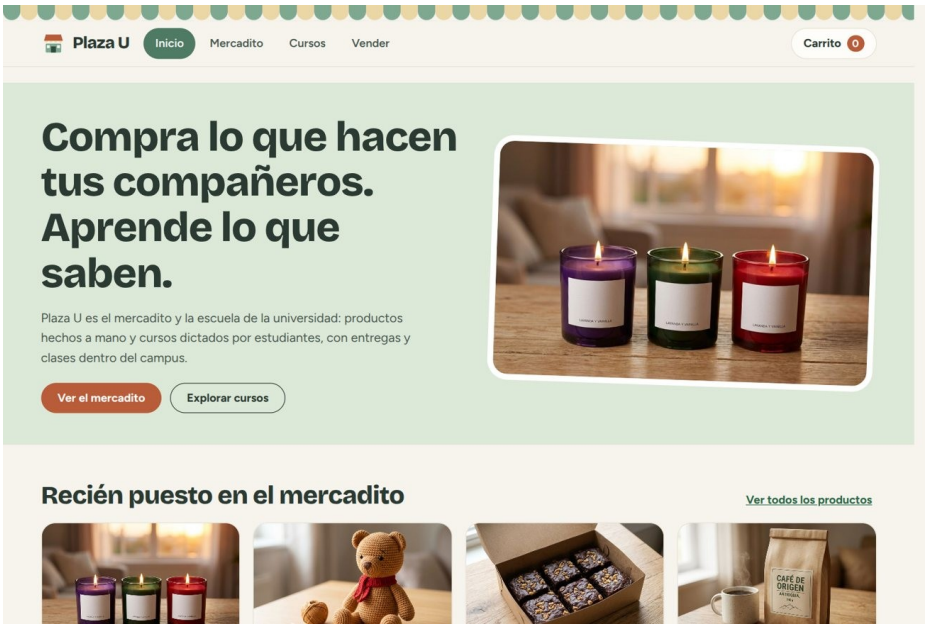


Figura 1. Página de inicio con la portada y los productos más nuevos.

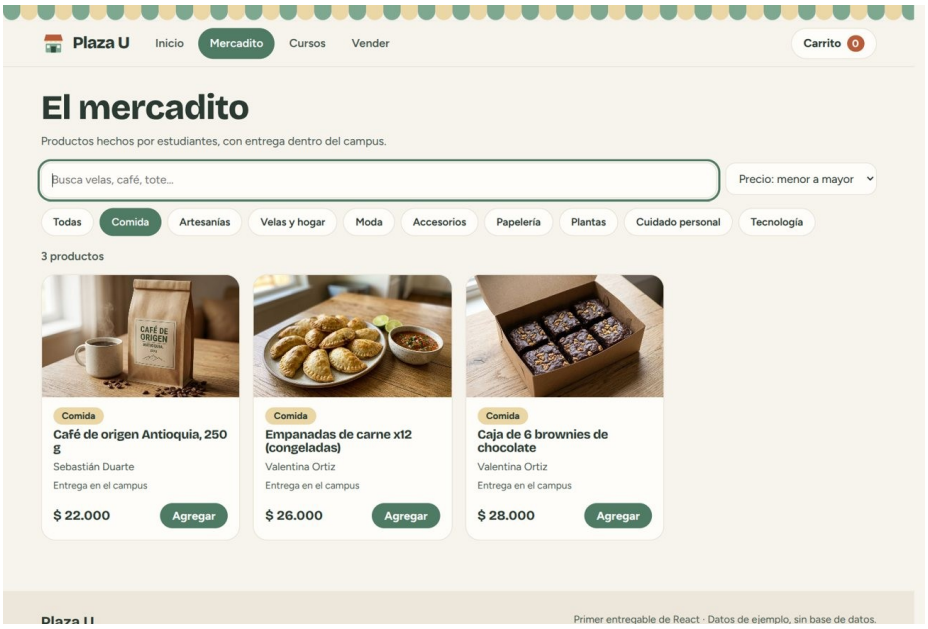


Figura 2. Mercadito filtrado por la categoría «Comida» y ordenado por precio.

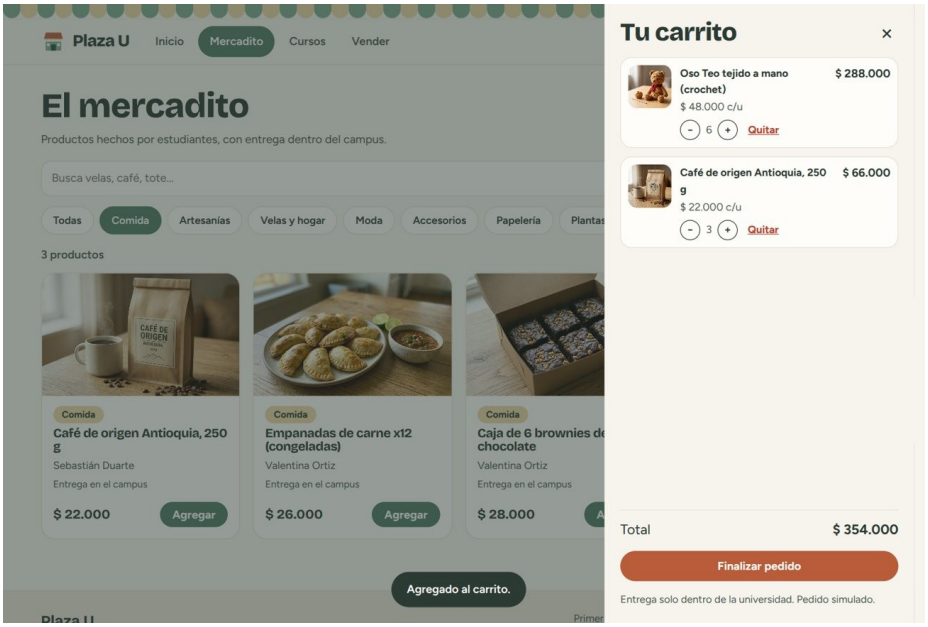


Figura 3. Carrito con dos productos, sus cantidades y el total.



Figura 4. Cursos filtrados por nivel, con una inscripción hecha y «Mis inscripciones».

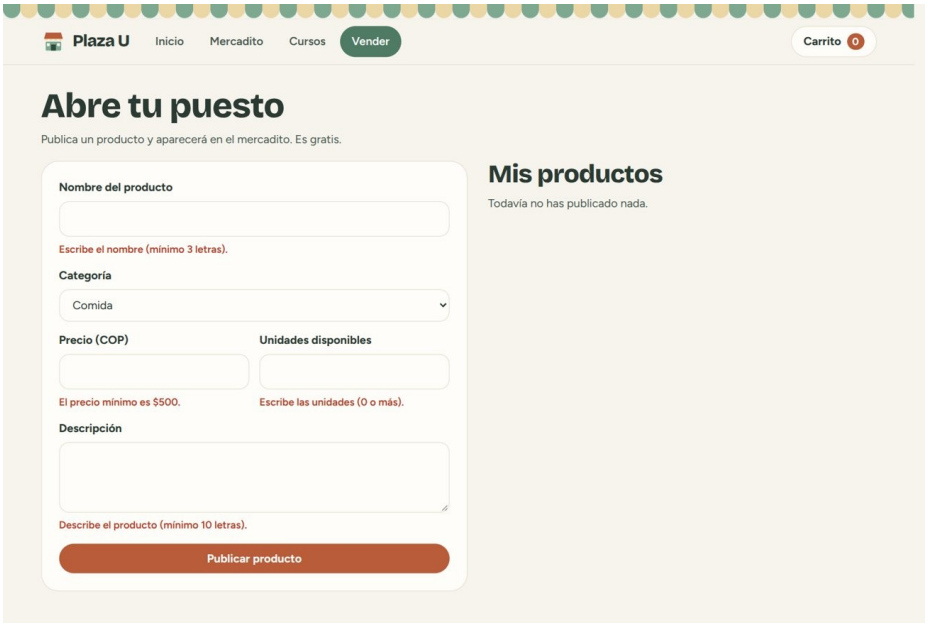


Figura 5. Formulario de «Vender» con los mensajes de validación.

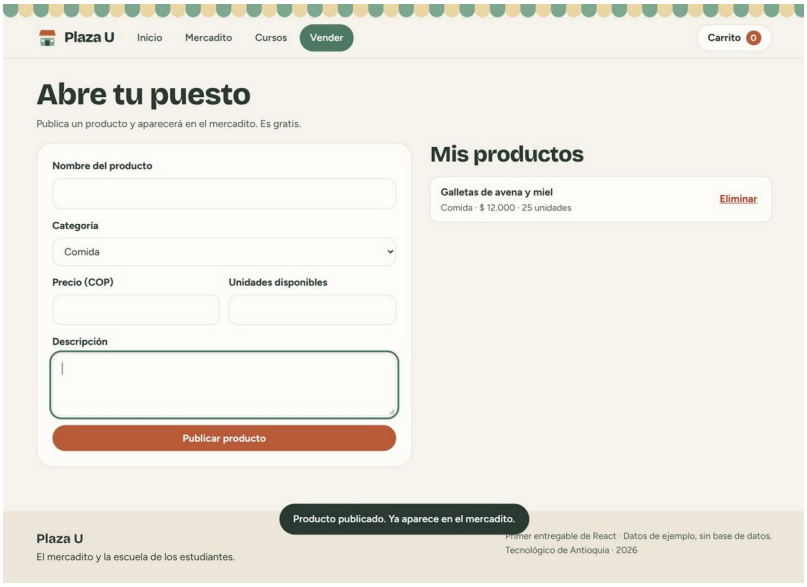


Figura 6. Producto publicado, visible en «Mis productos».



Figura 7. La interfaz en un celular (390 px de ancho).

9. Pruebas realizadas

Probamos la aplicación en un navegador real, con 30 comprobaciones que repiten lo que haría una persona: abrir cada sección, buscar, filtrar, ordenar, agregar al carrito hasta el límite del stock, quitar, finalizar, inscribirse, enviar el formulario vacío y con datos, eliminar y abrir la vista de celular. Todas pasaron y no hubo errores en la consola del navegador.

10. Cómo ejecutar el proyecto

```
npm install     # instala las dependencias
npm run dev     # abre la app en http://localhost:5173
npm run build   # genera la versión de producción
npm run lint    # revisa el código
```

Necesita Node.js 18 o superior. El repositorio está en GitHub: **[pegar aquí el enlace del repositorio]**.

11. Conclusiones y siguientes pasos

Con este entregable dejamos armada la base del frontend de Plaza U: una estructura de carpetas clara, componentes que se reutilizan, el estado concentrado en un solo lugar y las funciones principales (catálogo, carrito, cursos y publicación) funcionando con datos de ejemplo.

Lo siguiente es conectar la interfaz con una API y una base de datos para que la información se guarde de verdad, y agregar el registro de usuarios, los pedidos reales y las comisiones.

Esa versión completa ya la tenemos avanzada en otro desarrollo del proyecto; este entregable toma solo el frontend con datos simulados, como pide la entrega.

Referencias

React. Documentación oficial: react.dev

Vite. Guía oficial: vitejs.dev

MDN Web Docs. Array.prototype.map, filter y reduce: developer.mozilla.org